LAB 7 · LECTURE 7

Lecture 7 Lab: ChromaDB Collections and Queries

Vector Databases Course | Estimated time: 75 minutes | Platform: Google Colab (free)

Goal

You will build a complete Chroma collection, query it with metadata and text filters, see how the distance setting changes rankings, persist the data, run Chroma in server mode, and ingest the chunks from Lab 6.

- Create collections with an explicit embedding function and distance.
- Use add, upsert, get, query, update and delete, and see why add is not enough.
- Filter with where and where_document, and read the result structure.
- Compare l2, cosine and inner product on vectors of different lengths.
- Persist to disk and connect to a Chroma server with HttpClient.
- Trigger the common errors on purpose.

Setup

Create a new Colab notebook and run each block in its own cell, in order.

Cell 1: install and import

```
!pip install -q chromadb sentence-transformers pandas numpy

import os, json, time, shutil, subprocess
import numpy as np, pandas as pd
import chromadb
from chromadb.utils import embedding_functions as ef
print("chromadb", chromadb.__version__)
```

Part 1: A first collection

The same thirty help-center documents as Lab 5, now with metadata fields: topic, year, priority and language.

Cell 2: data

```
data = [
 ("account", "How to reset your account password"),
 ("account", "Change the email address on my profile"),
 ("account", "Delete my account permanently"),
 ("account", "Enable two-factor authentication"),
 ("account", "I forgot my login credentials"),
 ("account", "Update my display name"),
 ("billing", "Why was I charged twice this month"),
 ("billing", "Download my invoice as a PDF"),
 ("billing", "Update my credit card details"),
 ("billing", "How to cancel my subscription"),
 ("billing", "Refund policy for cancelled orders"),
 ("billing", "Apply a discount code at checkout"),
 ("shipping", "Where is my package right now"),
 ("shipping", "How long does international delivery take"),
 ("shipping", "Change the delivery address after ordering"),
 ("shipping", "Shipping costs for large items"),
 ("shipping", "My parcel arrived damaged"),
 ("shipping", "Track an order without an account"),
 ("technical", "The app crashes when I open settings"),
 ("technical", "Wi-Fi keeps disconnecting every few minutes"),
 ("technical", "Laptop battery drains too quickly"),
 ("technical", "My screen flickers after the update"),
 ("technical", "How to back up my phone"),
 ("technical", "Bluetooth headphones will not pair"),
 ("returns", "Start a return for a defective item"),
 ("returns", "How to exchange a product for a different size"),
 ("returns", "Is a return label free"),
 ("returns", "Return window for electronics"),
 ("returns", "Where to drop off my return"),
 ("returns", "Refund timeline after a return"),
]

topics = [t for t, _ in data]
texts  = [s for _, s in data]
ids    = [f"h{i}" for i in range(len(texts))]
metas  = [{"topic": topics[i], "year": 2022 + i % 4,
           "priority": 1 + i % 3, "lang": "en"} for i in range(len(texts))]
print(len(texts), "documents")
```

Cell 3: client, embedding function, collection

```
minilm = ef.SentenceTransformerEmbeddingFunction(model_name="all-MiniLM-L6-v2")

client = chromadb.EphemeralClient()
col = client.create_collection("help", embedding_function=minilm,
                               configuration={"hnsw": {"space": "cosine"}})
col.add(ids=ids, documents=texts, metadatas=metas)
print(col.count(), col.configuration["hnsw"]["space"])
```

Part 2: add versus upsert

Cell 4: what happens with an existing id?

```
col.add(ids=["h0"], documents=["CHANGED TEXT"])
print("after add   :", col.get(ids=["h0"])["documents"])

col.upsert(ids=["h0"], documents=["CHANGED TEXT"], metadatas=[metas[0]])
print("after upsert:", col.get(ids=["h0"])["documents"])

col.upsert(ids=["h0"], documents=[texts[0]], metadatas=[metas[0]])   # restore
```

YOUR TURN Which call changed the record and which did not? Write the consequence for an ingestion script that re-reads edited files.

Part 3: Querying

Cell 5: query and a helper that formats the results

```
def show(res):
    dist = np.array(res["distances"][0])
    return pd.DataFrame({"id": res["ids"][0], "distance": dist.round(3),
                         "similarity": (1 - dist).round(3),
                         "topic": [m["topic"] for m in res["metadatas"][0]],
                         "text": res["documents"][0]})

show(col.query(query_texts=["I can't sign in to my profile"], n_results=5))
```

YOUR TURN Why is similarity computed as 1 minus the distance here, and when would that formula be wrong? (Hint: check the collection's space.)

Part 4: Filters

Cell 6: where, $and, $in, where_document, get

```
q = ["when will my order get here"]

print(show(col.query(query_texts=q, n_results=5, where={"topic": "billing"})), "\n")

strict = {"$and": [{"topic": {"$in": ["shipping", "returns"]}},
                   {"year": {"$gte": 2024}}]}
print(show(col.query(query_texts=q, n_results=5, where=strict)), "\n")

print(show(col.query(query_texts=q, n_results=5,
                     where_document={"$contains": "order"})), "\n")

page = col.get(where={"priority": 3}, limit=5, include=["documents", "metadatas"])
print(page["ids"])
```

YOUR TURN The strict filter matches only a few records. How many results did you get with n_results=5, and why is it not an error? Add a $or filter and describe what changes.

Part 5: Distances on vectors of different lengths

Most text models return unit vectors, so all spaces agree. We scale each vector by a random factor to imitate a model that does not normalize, and supply the embeddings ourselves.

Cell 7: three collections, three spaces

```
E = np.array(minilm(texts))
rng = np.random.default_rng(0)
scaled = E * rng.uniform(0.5, 3.0, size=(len(E), 1))
qv = np.array(minilm(q))[0]

def make(name, space, vectors):
    c = client.create_collection(name, embedding_function=None,
                                 configuration={"hnsw": {"space": space}})
    c.add(ids=ids, embeddings=vectors.tolist(), documents=texts)
    return c

results = {}
for space in ["l2", "cosine", "ip"]:
    c = make(f"scaled_{space}", space, scaled)
    r = c.query(query_embeddings=[qv.tolist()], n_results=3)
    results[space] = r["documents"][0]
pd.DataFrame(results)
```

YOUR TURN Which space gives a different top-3 than the others on scaled vectors, and which is unaffected by the scaling? Connect your answer to Lecture 3.

Part 6: Persistence

Cell 8: write to disk, forget, reopen

```
if os.path.exists("chroma_lab"):
    shutil.rmtree("chroma_lab")

p = chromadb.PersistentClient(path="chroma_lab")
pc = p.get_or_create_collection("help", embedding_function=minilm,
                                configuration={"hnsw": {"space": "cosine"}})
pc.upsert(ids=ids, documents=texts, metadatas=metas)
print("files:", os.listdir("chroma_lab"))
del pc, p

p2 = chromadb.PersistentClient(path="chroma_lab")
pc2 = p2.get_collection("help", embedding_function=minilm)
print("reopened:", pc2.count(), "records")
```

Part 7: Server mode

Cell 9: start a server and connect with HttpClient

```
srv = subprocess.Popen(["chroma", "run", "--path", "chroma_server", "--port", "8000"],
                       stdout=subprocess.DEVNULL, stderr=subprocess.STDOUT)
time.sleep(8)

http = chromadb.HttpClient(host="localhost", port=8000)
print("heartbeat ok:", http.heartbeat() > 0)
hc = http.get_or_create_collection("help", embedding_function=minilm,
                                   configuration={"hnsw": {"space": "cosine"}})
hc.upsert(ids=ids, documents=texts, metadatas=metas)
print(hc.count())
show(hc.query(query_texts=["reset my password"], n_results=3))
```

YOUR TURN In server mode, which component computes the embeddings: the client or the server? Test by reading the documentation or the behavior, and write what that means for deployment.

Part 8: Ingest your Lab 6 chunks

Cell 10: records from chunks.jsonl

```
if os.path.exists("chunks.jsonl"):
    records = [json.loads(line) for line in open("chunks.jsonl")]
else:
    print("chunks.jsonl not found; using the help documents instead")
    records = [{"id": f"demo:{i}", "text": t,
                "metadata": {"source": "demo", "page": 1 + i // 10, "chunk": i}}
               for i, t in enumerate(texts)]

paper = client.get_or_create_collection("paper", embedding_function=minilm,
                                        configuration={"hnsw": {"space": "cosine"}})
paper.upsert(ids=[r["id"] for r in records],
             documents=[r["text"] for r in records],
             metadatas=[r["metadata"] for r in records])
print(paper.count(), "chunks")

res = paper.query(query_texts=["Which optimizer was used for training?"], n_results=3,
                  where={"page": {"$lte": 10}})
for d, m in zip(res["documents"][0], res["metadatas"][0]):
    print(m["page"], "|", d[:100].replace("\n", " "))

paper.delete(where={"source": records[0]["metadata"]["source"]})
print("after delete by source:", paper.count())
```

YOUR TURN Restore the collection with one call. Then change one document's text, upsert it again, and verify that the old version is gone.

Part 9: Break it on purpose

Cell 11: common errors

```
def attempt(label, fn):
    try:
        fn()
        print(label, "-> no error")
    except Exception as e:
        print(label, "->", type(e).__name__, ":", str(e)[:70])

attempt("wrong dimension", lambda: col.add(ids=["z"], embeddings=[[0.1] * 5],
                                           documents=["x"]))
attempt("nested metadata", lambda: col.add(ids=["y"], documents=["x"],
                                           metadatas=[{"bad": {"a": 1}}]))
attempt("duplicate collection", lambda: client.create_collection("help"))
attempt("unknown operator", lambda: col.query(query_texts=["x"],
                                              where={"topic": {"$like": "a"}}))
srv.terminate()
```

YOUR TURN For each error, write the cause and the fix. Which of them would you catch only in production, and how would you test for it earlier?

Reflection questions

- Why does the lab create the collection with an explicit embedding function and space instead of relying on defaults?
- A teammate stores three embedding models in one collection to compare them. What goes wrong, and what do you suggest?
- Your application needs to filter by customer on every query. Compare a customer metadata field with one collection per customer.
- When would you move from Chroma to another system from Lecture 5? List two concrete signals.

Deliverable

Submit the executed notebook, the answers to each YOUR TURN box, and short answers to the reflection questions.

Troubleshooting

- **Collection already exists:** restart the runtime, or use get_or_create_collection and a new client.
- **Server does not start:** wait longer in Cell 9, or run `!chroma run --path chroma_server` in a terminal tab.